

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA  
TECHNICAL UNIVERSITY OF MOLDOVA  
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS  
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS  
LABORATORY WORK #9 - PART 1

---

# Automatic Temperature Control with ON-OFF Hysteresis and Assisted Cooling

---

*Author:*

Sava LUCHIAN  
std. gr. FAF-233

*Verified:*

Alexei MARTINIUC

Chisinau 2026

# Technical Task

## Purpose of the Laboratory

The purpose of this laboratory is to design and validate a modular embedded application on **Arduino Uno** that performs **ON-OFF temperature control with hysteresis**. The process value is measured by a **10k NTC thermistor**, the setpoint is adjusted from a **potentiometer**, heating is produced by a resistor, and excessive temperature is mitigated by a **DC fan driven through L293D**. Runtime values are shown on LCD and sent through **STDIO** using `printf()`.

## Objectives

1. Implement a modular control system with separated sensor, input, control, actuator, display, and application layers.
2. Implement ON-OFF control with hysteresis for the heating stage.
3. Add a gradual cooling stage using an L293D-driven fan when the temperature exceeds the upper comfort band.
4. Use sequential periodic tasks with justified recurrences.
5. Display the relevant values locally on LCD and send CSV telemetry through serial.

## Implemented Variant

The implemented solution follows the thermal-control variant on **Arduino Uno (ATmega328P)**:

- **Sensor input:** analog temperature acquisition from a thermistor divider on A0.
- **Setpoint input:** potentiometer on A1 mapped to 20.0–80.0 °C.
- **Controller:** ON-OFF with hysteresis half-band of 1.0 °C.
- **Heating actuator:** resistor switched from D8.
- **Cooling actuator:** DC motor fan controlled through L293D using PWM.
- **Reporting:** LCD + `printf()` CSV in the format `setpoint,temperature,heater,fan`.

# 1 Domain Analysis

## 1.1 Technological Context and Application Domain

ON-OFF control is suitable for simple thermal systems where a binary heater is sufficient to keep the temperature around a configurable setpoint. Hysteresis avoids frequent switching around the reference point and therefore reduces actuator chatter.

In this implementation, the system is extended with assisted cooling. When the measured temperature exceeds the upper side of the control band, a fan is enabled through an L293D driver and its speed increases gradually with temperature. This hybrid structure keeps the heating logic simple while improving thermal safety and response.

## 1.2 Hardware Components and Justification

| Component                     | Qty | Role in the application                                                           |
|-------------------------------|-----|-----------------------------------------------------------------------------------|
| Arduino Uno (ATmega328P)      | 1   | Main MCU for acquisition, control, actuation, LCD handling, and serial reporting. |
| NTC thermistor 10k            | 1   | Analog temperature sensing element.                                               |
| Fixed resistor 10k            | 1   | Forms the thermistor voltage divider on A0.                                       |
| Potentiometer 10k             | 1   | User-adjustable setpoint source on A1.                                            |
| Heater resistor 220 $\Omega$  | 1   | Resistive thermal source controlled from D8.                                      |
| L293D motor driver            | 1   | Drives the cooling fan from Uno digital pins.                                     |
| DC motor with fan             | 1   | Active cooling output with gradual speed increase.                                |
| LCD1602 + support resistors   | 1   | Local status display for setpoint, temperature, heater, and fan state.            |
| USB serial terminal / plotter | 1   | Receives <code>printf()</code> telemetry.                                         |

Table 1: Hardware components used in the final Lab 9 implementation

## 1.3 Software Components and Technologies

- **PlatformIO + VS Code:** project build and dependency management.
- **Arduino framework:** GPIO, ADC, timing, PWM, serial, LCD support.
- **AVR-libc STDIO:** stdout redirected to serial through `fdev_setup_stream`.

- **LiquidCrystal**: 1602 LCD driver in 4-bit mode.
- **Wokwi**: simulation environment for the Uno wiring and runtime checks.
- **PlantUML**: source language for the diagrams included in the report.

## 2 Conceptualisation and Design

### 2.1 Technical Requirements

| ID      | Category   | Requirement                                                                          | Type      |
|---------|------------|--------------------------------------------------------------------------------------|-----------|
| R-F-01  | Sensor     | System shall read temperature from a thermistor divider.                             | Func.     |
| R-F-02  | Setpoint   | System shall read setpoint from potentiometer on A1.                                 | Func.     |
| R-F-03  | Control    | System shall implement ON-OFF hysteresis for the heater.                             | Func.     |
| R-F-04  | Cooling    | System shall drive a fan through L293D when temperature is too high.                 | Func.     |
| R-F-05  | Cooling    | Fan speed shall increase gradually as temperature rises above the cooling threshold. | Func.     |
| R-F-06  | Display    | LCD shall show setpoint, temperature, heater state, and fan level.                   | Func.     |
| R-F-07  | Reporting  | Runtime values shall be sent through <code>printf()</code> as CSV.                   | Func.     |
| R-NF-01 | Timing     | The application shall use justified sequential periodic tasks.                       | Non-func. |
| R-NF-02 | Modularity | Code shall be split into reusable modules.                                           | Non-func. |
| R-NF-03 | Platform   | Firmware shall fit Arduino Uno resource limits.                                      | Non-func. |

Table 2: Technical requirements for the final Lab 9 system

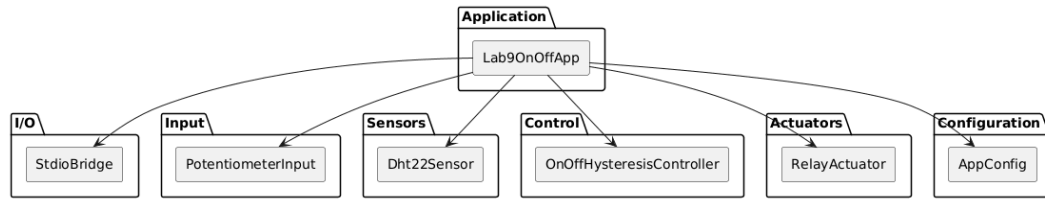


Figure 1: Layered software architecture of the final Lab 9 system

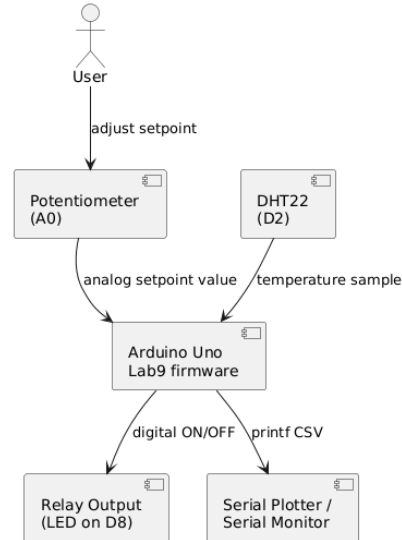


Figure 2: Main hardware and software interactions

## 2.2 Architectural Design

### 2.2.1 System Structural Architecture

### 2.2.2 HW/SW Interface and Component Interaction

### 2.2.3 Control Pipeline

| Stage                | Purpose                                                                              |
|----------------------|--------------------------------------------------------------------------------------|
| Setpoint acquisition | Reads the potentiometer and maps it to a temperature reference.                      |
| Feedback acquisition | Reads thermistor ADC value and converts it to degrees Celsius.                       |
| Heater control       | Applies ON-OFF hysteresis thresholds to the heating resistor.                        |
| Cooling control      | Computes fan target speed above the high-temperature band and ramps speed gradually. |
| Presentation         | Updates LCD and prints CSV telemetry through serial.                                 |

Table 3: Control pipeline used in the final Lab 9 implementation

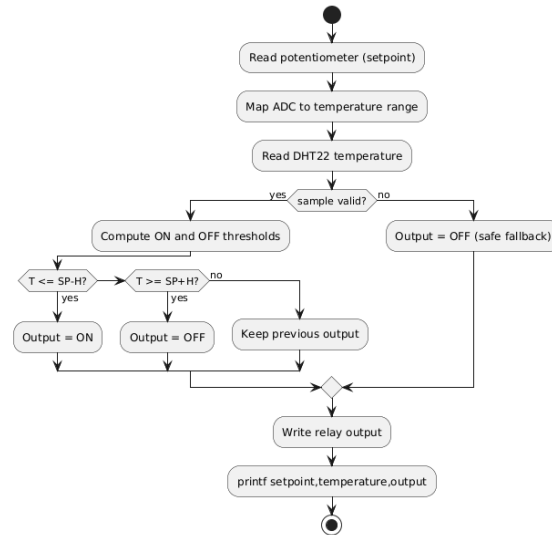


Figure 3: Combined heating and cooling control pipeline

### 2.2.4 Main Loop Timing Design

| Activity           | Period | Trigger          | Handler              |
|--------------------|--------|------------------|----------------------|
| Setpoint update    | 100 ms | millis() elapsed | runSetpointTask()    |
| Acquisition update | 100 ms | millis() elapsed | runAcquisitionTask() |
| Control update     | 250 ms | millis() elapsed | runControlTask()     |
| Report update      | 500 ms | millis() elapsed | runReportTask()      |
| Fan ramp update    | 100 ms | millis() elapsed | fan actuator tick()  |

Table 4: Sequential periodic timing model for Lab 9

## 2.3 Behavioural Modelling

### 2.3.1 Application Tick Flowchart

### 2.3.2 Control Decision Flowchart

### 2.3.3 Sequence Diagram – One Control Cycle

### 2.3.4 Hysteresis State Diagram

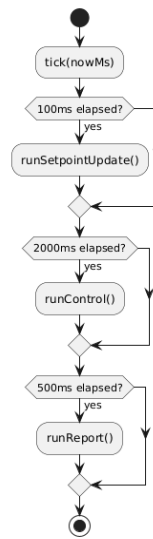


Figure 4: Flowchart of the sequential application loop

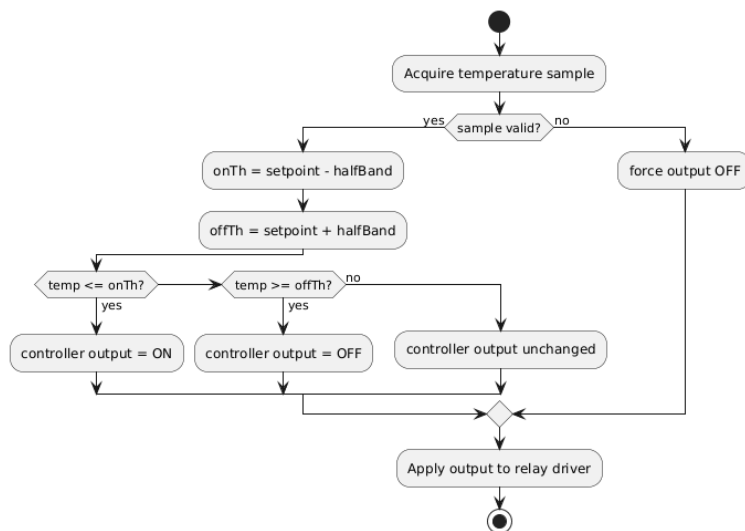


Figure 5: Flowchart of the ON-OFF hysteresis decision logic

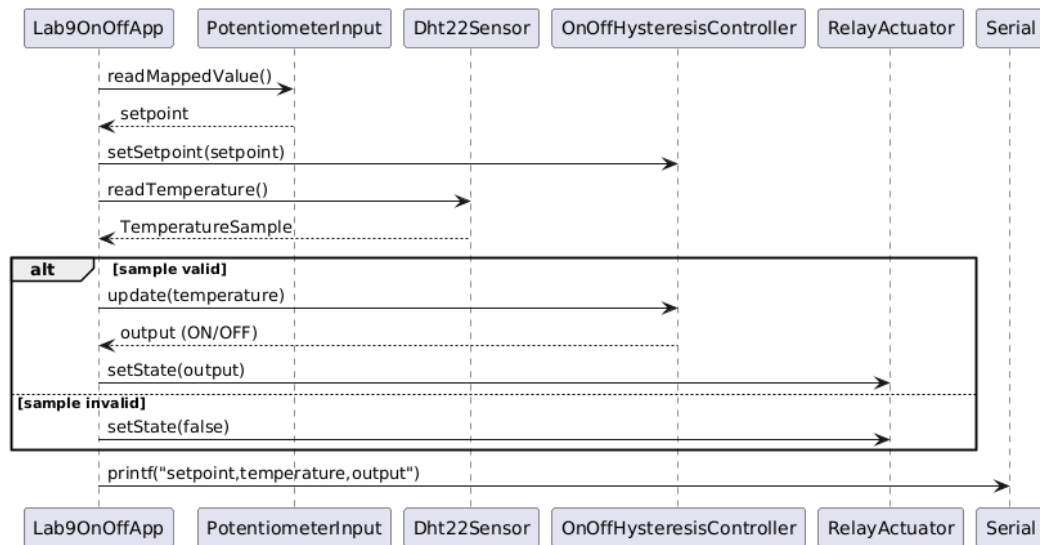


Figure 6: Sequence diagram of one acquisition-control-report cycle

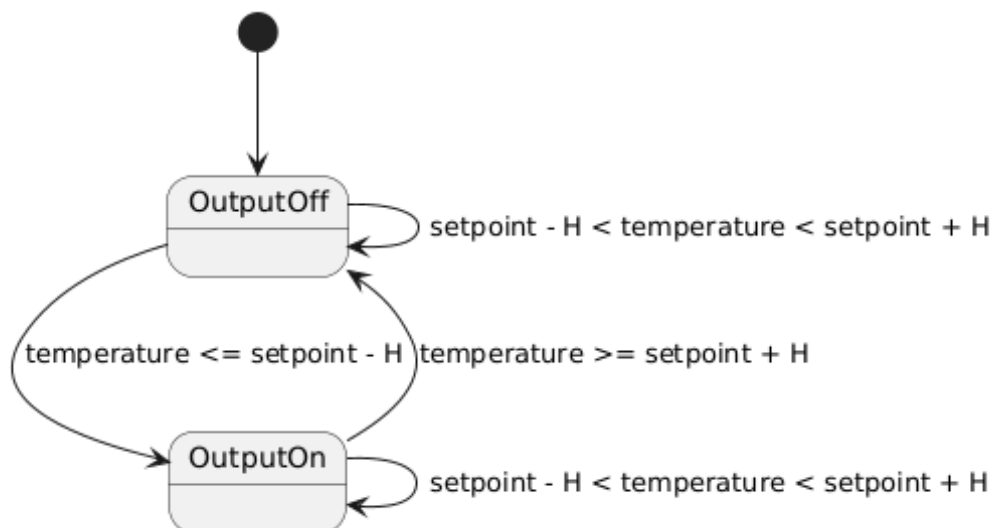


Figure 7: Heater state transitions with hysteresis memory



### 3 Hardware and Software Implementation

#### 3.1 Electrical Schematic and Implemented Setup

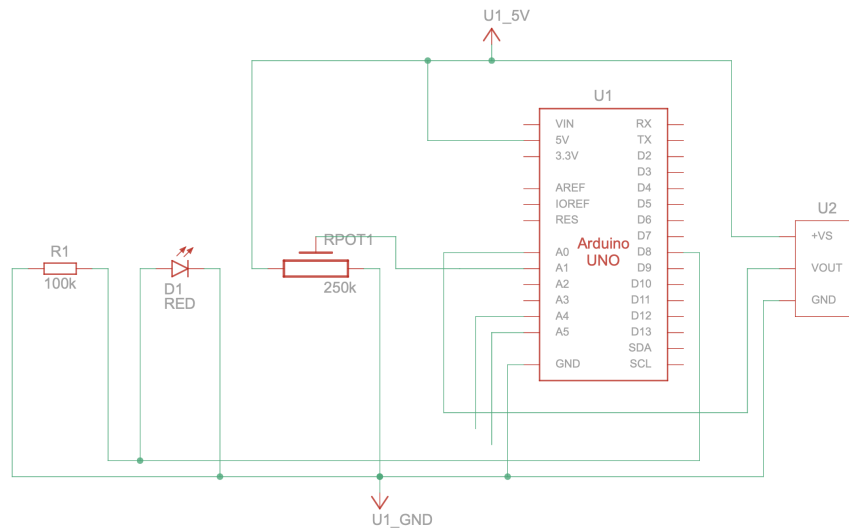


Figure 8: Electrical circuit schematic of the final Lab 9 implementation

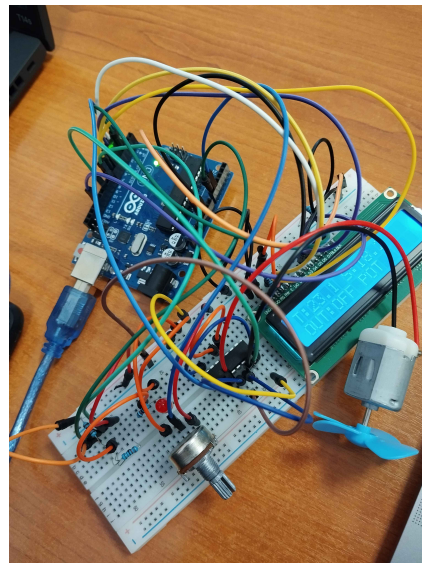


Figure 9: Hardware setup used for the final Lab 9 implementation

#### Pin assignment:

- A0 – thermistor divider node
- A1 – potentiometer signal
- D8 – heater resistor output
- D2 – L293D IN1

- D10 – L293D IN2
- D11 – L293D EN1,2 (PWM)
- D7 – LCD RS
- D6 – LCD E
- D5 – LCD D4
- D4 – LCD D5
- D3 – LCD D6
- D13 – LCD D7
- D0/D1 – USB serial UART for STDIO output

## 3.2 Project Directory Structure

```
lab9/
|-- platformio.ini
|-- diagram.json
|-- wokwi.toml
|-- src/
|   |-- main.cpp
|   |-- config/
|   |   '-- AppConfig.h
|   |-- sensors/
|   |   '-- ThermistorSensor.h/.cpp
|   |-- inputs/
|   |   '-- PotentiometerInput.h/.cpp
|   |-- control/
|   |   '-- OnOffHysteresisController.h/.cpp
|   |-- actuators/
|   |   |-- HeaterActuator.h/.cpp
|   |   '-- L293dFanActuator.h/.cpp
|   |-- io/
|   |   |-- CommandInterface.h/.cpp
|   |   |-- LcdDisplay.h/.cpp
|   |   '-- StdioBridge.h/.cpp
|   '-- app/
|       '-- Lab9ControlApp.h/.cpp
'-- report/
```

```

|-- main.tex
|-- hardware_photo.jpg
'-- plantuml/

```

### 3.3 Software Module Descriptions

- **ThermistorSensor**: converts ADC readings to voltage, resistance, and temperature.
- **PotentiometerInput**: maps ADC value to the configurable setpoint range.
- **OnOffHysteresisController**: implements thresholded heater logic with memory.
- **HeaterActuator**: drives the heating resistor from a digital pin.
- **L293dFanActuator**: drives and ramps the cooling fan through L293D using PWM.
- **CommandInterface**: handles serial commands such as `help` and `status`.
- **LcdDisplay**: presents local runtime status on the 1602 LCD.
- **Lab9ControlApp**: orchestrates sequential periodic tasks.

## 4 Testing and Validation

### 4.1 Build Results

| Environment | Flash used | Flash % | RAM used | RAM % |
|-------------|------------|---------|----------|-------|
| uno         | 11 430 B   | 35.4%   | 788 B    | 38.5% |

Table 5: PlatformIO build resource usage for the final Lab 9 firmware

## 4.2 Test Scenarios and Execution Results

| ID   | Scenario              | Observed result                                                                  | Result |
|------|-----------------------|----------------------------------------------------------------------------------|--------|
| T-01 | Startup and build     | Firmware builds and starts correctly on Uno profile.                             | PASS   |
| T-02 | Setpoint sweep        | Potentiometer changes setpoint continuously across the configured range.         | PASS   |
| T-03 | Low temperature case  | When $T \leq SP - H$ , the heater turns ON.                                      | PASS   |
| T-04 | High temperature case | When $T \geq SP + H$ , the heater turns OFF.                                     | PASS   |
| T-05 | Deadband behaviour    | Inside the hysteresis band, heater state is preserved.                           | PASS   |
| T-06 | Fan start threshold   | Above $SP + 1.0^\circ C$ , the fan begins to spin.                               | PASS   |
| T-07 | Fan ramp behaviour    | Fan speed increases progressively as temperature rises.                          | PASS   |
| T-08 | LCD/serial output     | LCD and CSV output reflect current setpoint, temperature, heater, and fan state. | PASS   |
| T-09 | Resource limits       | Flash and RAM remain below Uno limits.                                           | PASS   |

Table 6: Test execution summary for the final Lab 9 version

## 4.3 Representative Serial Output

```

35.0, 24.8,1,0
35.0, 29.2,0,0
35.0, 38.4,0,26
35.0, 42.2,0,67
35.0, 45.1,0,100

```

The four channels represent setpoint, measured temperature, heater state, and fan speed percentage. This format is directly suitable for Arduino Serial Plotter style monitoring.

## General Conclusions

Laboratory Work #9 was completed as a modular automatic temperature-control system for Arduino Uno. The final implementation satisfies the required control structure and additionally introduces an assisted cooling stage for safer thermal management.

Key outcomes:

1. The thermistor-based thermal feedback loop successfully implements ON-OFF control with hysteresis.
2. The potentiometer provides a simple and responsive real-time setpoint interface.
3. The L293D-driven fan improves the thermal response by gradually increasing cooling effort above the hot region.
4. The firmware remains lightweight enough for Arduino Uno while preserving modular structure and clear runtime observability.

## Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code completion and refactoring assistance.
- **LLM assistance** – LaTeX drafting, editing support, and technical structure refinement.

All generated content was manually reviewed and adapted to the implemented project.

## Bibliography

1. Arduino Official Reference. <https://www.arduino.cc/reference/en/>
2. AVR-libc User Manual. <https://www.nongnu.org/avr-libc/user-manual/>
3. PlatformIO Documentation. <https://docs.platformio.org/en/latest/>
4. Wokwi Simulator Documentation. <https://docs.wokwi.com/>
5. LiquidCrystal Library Reference. <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>
6. Lab 9 theory and task notes on ON-OFF control with hysteresis. Technical University of Moldova, 2026.

## Annex – Source Code and PlantUML Sources

The full source code is hosted on GitHub:

**Repository URL:**

<https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab9>

PlantUML source files for the report diagrams are stored in:

[lab9/report/plantuml/](#)